



# CONCORRÊNCIA E MULTITHREADING

AULA 08 - VIRTUAL THREADS



***“...to use virtual threads effectively, there is more **unlearning** than learning to be done.”***  
**- Brian Goetz**



# Como funciona uma Thread na JVM?

# Threads no Java até então - Platform Threads

- `java.lang.Thread`
- É um wrapper em cima das threads de SO
- Threads de SO suportam todas as linguagens (nível por baixo)
- Alto consumo de RAM
- Troca de tasks em execução é feita no nível do kernel
- Há subutilização de hardware
- Para aumentar o throughput é necessário investir em hardware
- **Platform Threads são recursos (como memória, CPU, disco, etc)**

Porque esse modelo gera  
limitação de throughput?

# Little's Law

***“... (the) average number  $L$  of customers in a stationary system is equal to the ...average effective arrival rate  $\lambda$  multiplied by the average time  $W$  that a customer spends in the system”***

$$\underset{\text{Concorrência}}{L} = \underset{\text{Throughput}}{\lambda} \underset{\text{Duração (Latência)}}{W}$$

# Little's Law

$$\underset{\text{Concorrência}}{\mathbf{L}} = \underset{\text{Throughput}}{\boldsymbol{\lambda}} \underset{\text{Latência/Duração}}{\mathbf{W}}$$

$$\text{Concorrência} = \# \text{Threads} \times \text{Tempo/Thread}$$

# Little's Law

$$\underset{\text{Concorrência}}{L} = \underset{\text{Throughput}}{\lambda} \underset{\text{Latência/Duração}}{W}$$

## Modelo Thread-Per-Request

- Modelo de execução das Platform Threads
- Cada request consome uma thread durante toda sua duração
- A thread fica bloqueada durante todo o tempo de execução da request
- Quanto maior o número de threads possíveis de serem executadas, maior o throughput
- Diretamente ligado ao hardware disponível
- $L = \text{\#threads}$  (1 pra 1)

Se temos um problema claro de concorrência devido a esse modelo de thread, como resolvê-lo?

# Concorrência



- Múltiplas tarefas independentes que competem pelos mesmos recursos disponíveis
- Medida de performance: throughput (tarefa/unidade de tempo)

# Paralelismo



- Dividir uma tarefa em tarefas menores que utilizam múltiplos recursos disponíveis
- Medida de performance: latência (tempo)

# Solução até então: Programação Assíncrona

- Opção para evitar o desperdício de hardware
- Quando há espera por algum IO, ao invés de bloquear a thread, ela é “devolvida” ao pool e pode ser usada por outras requests
- O número de requests que podem ser atendidas é maior que o número de threads disponíveis
- O L (=concorrência) não é limitado pelo número de threads
- Problemas:
  - Não é um estilo de programação muito “intuitivo”
  - Requer um set específico de APIs
  - Grande impacto na observabilidade (troubleshooting, debug, profiling, etc)
- Porque? Porque toda a plataforma Java é organizada em threads:
  - Exceptions: contexto é fornecido através de uma sequência de chamadas de threads;
  - Debugger: passos a passo da execução de uma thread
  - Profiler (JFR): agrupa eventos em torno de threads
- Tudo isso é “perdido” quando utilizados uma abordagem assíncrona

Ou seja...



# Síncrono



- A programação é mais simplificada
- Interface mais intuitiva com a plataforma
- Potenciais problemas de escalabilidade
- Devs felizes
- Hardware triste

# Assíncrono



- Código mais difícil de ler e manter
- Boa sorte com debug
- Tende a escalar melhor
- Devs tristes
- Hardware feliz

E se existisse algo que fosse o melhor dos dois mundos?

- 1) código fácil de escrever
- 2) que fizesse o melhor uso do hardware

# Virtual Threads

- JEP 425 (Preview no JDK 19): <https://openjdk.org/jeps/425>
- JEP 436 (Second Preview no JDK 20): <https://openjdk.org/jeps/436>
- **JEP 444 (Final no JDK 21): <https://openjdk.org/jeps/444>**
- 5 anos de desenvolvimento
- Parte do Project Loom: <https://cr.openjdk.java.net/~rpressler/loom/Loom-Proposal.html>
- Visa permitir que desenvolvedores Java:
  - Escrevam aplicações que façam o melhor uso do hardware disponível:
    - Através de um código simples de ler e manter
    - Compatível com as APIs e ferramentas já existentes



# Virtual Threads

- `java.lang.Thread` - mesma API
- Não são um wrapper em cima da threads de SO
- 100% implementadas e executadas na JVM
- O runtime do Java implementa threads de modo mais eficiente que as threads de SO, porque ele sabe exatamente como uma aplicação Java funciona
- Por exemplo, as virtual threads podem usar o recurso resizable stacks, algo que está indisponível nas threads de SO
- “Debaixo dos panos”, a JVM aloca pouquíssimas threads (**carrier thread**) para alocar um número gigantesco de virtual threads
- Ou seja:
  - Programação Java tradicional
  - Melhor utilização dos recursos disponíveis (“near-optimal hardware utilization”)
  - Dev feliz e hardware feliz



**Logo, a saída para um melhor throughput não foi a adoção de um estilo de programação diferente, mas sim fazer com que a thread deixasse de ser um recurso (torná-la mais “barata”)**

# O que muda no meu código?



```
public class Demo {  
  
    public static void main(String[] args) throws Exception {  
        new Demo().run();  
    }  
  
    1 usage  
    void run() throws Exception {  
        var thread = Thread.startVirtualThread(  
            (() -> System.out.println("HELLO, THREAD!"))  
        );  
        thread.join();  
    }  
  
}
```

```
public class Demo {

    public static void main(String[] args) throws Exception {
        new Demo().runExecutor();
    }

    void runExecutor() throws Exception{

        for(;;){
            long start = System.currentTimeMillis();
            try(var executor = Executors.newVirtualThreadPerTaskExecutor()){

                for (int i=0; i < 100_000; i++){
                    executor.submit(() -> {
                        Thread.sleep(2000);
                        return null;
                    });
                }
            }
            long end = System.currentTimeMillis();

            System.out.println(end - start + " ms");
        }
    }
}
```

# Como usar Virtual Threads

- Importante: virtual threads executam *\*qualquer\** código Java
- Mas para conseguir alcançar a escalabilidade que elas oferecem, você pode dar uma força
- Vale a sabedoria de vó: "*muito ajuda quem não atrapalha, meu filho...*"
- Se você sempre manipulou threads diretamente, talvez precise mudar algumas práticas



# 4 passos para adotar Virtual Threads

1. Use as boas e velhas blocking APIs
2. Uso de thread pools = poucas threads / Uso de thread per task = um monte de thread
3. Fuja do Pinning
4. Cuidado com casos de monopolização

# 1) Use as boas e velhas blocking APIs

```
CompletableFuture.supplyAsync(info::getUrl, pool)
    .thenCompose(url -> getBodyAsync(url, HttpResponse.BodyHandlers.ofString()))
    .thenApply(info::findImage)
    .thenCompose(url -> getBodyAsync(url, HttpResponse.BodyHandlers.ofByteArray()))
    .thenApply(info::setImageData)
    .thenAccept(this::process)
    .exceptionally(t -> { t.printStackTrace(); return null; });
```



```
try {
    String page = getBody(info.getUrl(), HttpResponse.BodyHandlers.ofString());
    String imageUrl = info.findImage(page);
    byte[] data = getBody(imageUrl, HttpResponse.BodyHandlers.ofByteArray());
    info.setImageData(data);
    process(info);
} catch (Exception ex) {
    t.printStackTrace();
}
```

## 2) Uso de thread pools = poucas threads / Uso de thread per task = um monte de thread

- Não transforme uma SO thread em uma virtual thread (zero ganho)
- Transforme uma task em uma virtual thread
  - Task = unidade de trabalho
  - Podem ser instâncias de Runnable ou Callable, por exemplo
- Não compartilhe uma virtual thread entre várias tasks
- **Virtual threads não são recursos, são objetos de lógica de negócio**

```
TimerTask task1 = new MyTimerTask("task1");
TimerTask task2 = new MyTimerTask("task2");

try(var executor = Executors.newVirtualThreadPerTaskExecutor()){
    executor.submit(task1);
    executor.submit(task2);
}
```

### 3) Fuja do Pinning

- Pinning é quando:
  - Um processo leva muito tempo para ser concluído (block);
  - Roda em modo syncrono e;
  - É executado com muita frequência
- Os 3 tem que estar presentes ao mesmo tempo, se não há pinning
- Essa combinação pode impactar a escalabilidade das virtual threads
- Para resolver:

```
lock lock  
k.lo
```

```
entrantLock();
```

```
igLongSynchronizedAndFree
```

### 3) Fuja do Pinning

- Importante: não precisa sincronizando todos os blocos \*sync do objeto
- Em versões futuras, esse erro deve ser resolvido servindo a finalidade do `entrantLock`

### ~~3) Fuja do Pinning~~

- O pinning foi resolvido no Java 24 (JEP 491)



## 4) Cuidado com casos de monopolização

- Não use virtual threads para executar processos de longa duração (CPU-bound workload)
- Esse tipo de workload precisa aguardar o término do I/O, e por isso não posso fazer swaps rápidos entre threads
- Ao fazer isso, ele bloqueia um core de CPU até a conclusão do I/O; ou seja, ele monopoliza a carrier thread
- O que leva a criação de outras carrier threads, resultando não só em degradação de performance, como até mesmo à exaustão de memória

**E seu eu quisesse limitar a  
concorrência a um  
recurso acessado por  
virtual threads?**

# Limite a concorrência usando um Semaphore

```
Semaphore sem = new Semaphore(permits: 10);  
sem.acquire();  
try{  
    return somethingVeryLimited();  
} finally{  
    sem.release();  
}
```

# **Futuro: Structured Concurrency**



# Futuro: Structured Concurrency

- Visa simplificar a programação concorrente em Java, tratando grupos de tasks que executam em diferentes threads como se fossem uma única unidade de trabalho
- Facilita o tratamento de erros e cancelamento de tasks filhas de modo coordenado
- Aumenta a previsibilidade e o controle da execução do código
- Facilita a observabilidade
- 5th Preview no Java 24

# Futuro: Structured Concurrency

- Visa simplificar a programação concorrente em Java, tratando grupos de tasks que executam em diferentes threads como se fossem uma única unidade de trabalho
- Facilita o tratamento de erros e cancelamento de tasks filhas de modo coordenado
- Aumenta a previsibilidade e o controle da execução do código
- Facilita a observabilidade
- 5th Preview no Java 24



# Futuro: Structured Concurrency

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {  
    Subtask<String> subtask1 = scope.fork() -> fetchDataFromApi1();  
    Subtask<String> subtask1 = scope.fork() -> fetchDataFromApi2();  
  
    scope.join();  
    scope.throwIfFailed();  
  
    String result1 = subtask1.get();  
    String result2 = subtask1.get();  
}
```

# Virtual Threads no Quarkus

# Virtual Threads no Quarkus

```
@GET
@RunOnVirtualThread
public String process() {
    // Runs on a virtual thread because the
    // method uses the @RunOnVirtualThread annotation.

    // `service` is a rest client, it executes an I/O operation
    var response = service.greetings(); // Blocking, but this time, it
    // does neither block the carrier thread
    // nor the OS thread.
    // Only the virtual thread is blocked.
    return response.toUpperCase();
}
```

# Referências

- Virtual Threads: New Foundations for High-Scale Java Applications (Brian Goetz, Daniel Briant): <https://infoq.com/articles/java-virtual-threads/>
- The Age of Virtual Threads (Ron Pressler, Alan Bateman): <https://youtu.be/YQ6EpIk7KgY>
- JEP 444: Virtual Threads (Final no JDK 21): <https://openjdk.org/jeps/444>
- Use Virtual Threads in Rest applications: <https://quarkus.io/guides/rest-virtual-threads>
- Virtual Thread support reference: <https://quarkus.io/guides/virtual-threads>



**ATÉ A PRÓXIMA!**